Computing Methods for Experimental Physics and Data Analysis

Introduction

L. Baldini, F. A. Di Bello, G. Lamanna, F. Lizzi, A. Manfreda, A. Retico, A. Rizzi

luca.baldini@pi.infn.it

Università and INFN–Pisa

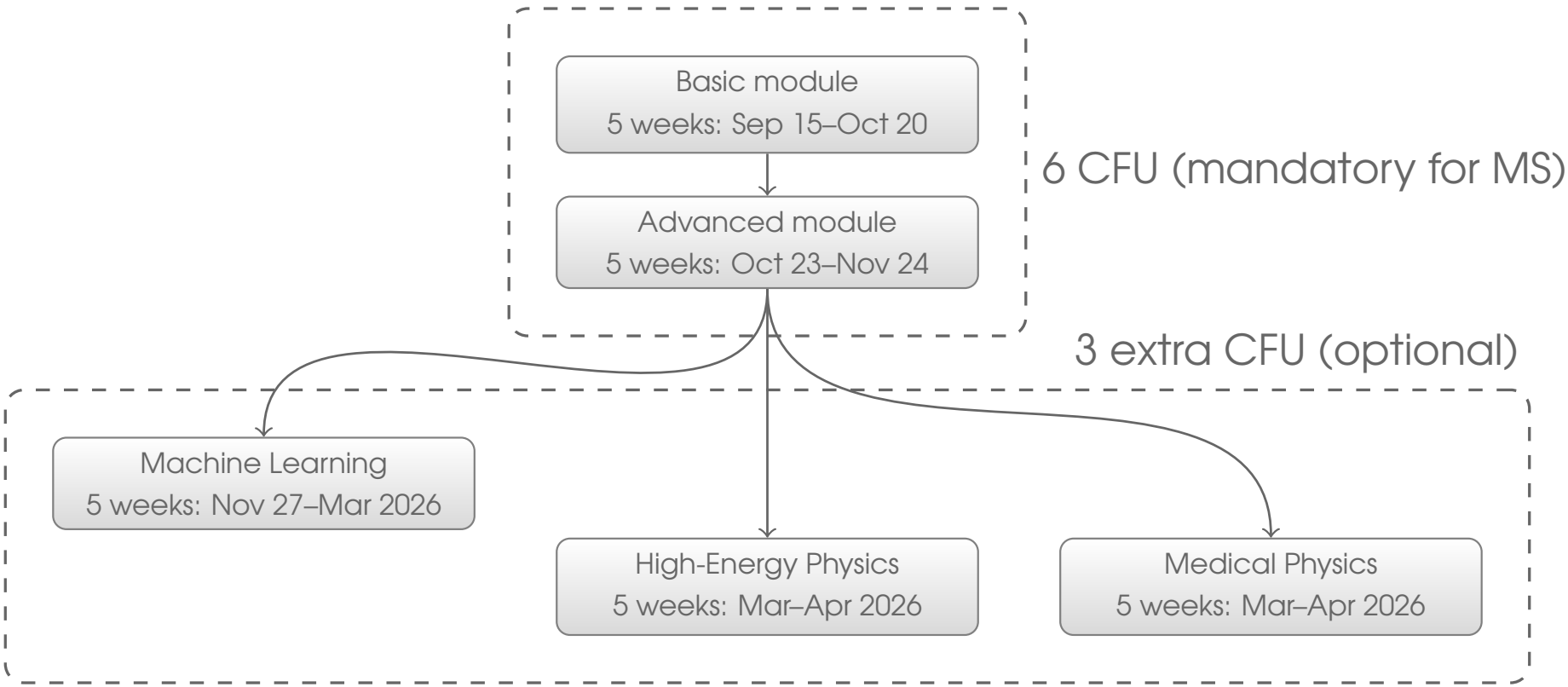
September 15, 2026



Goals and prerequisites

- What is this all about?
  - Collaborative code development and best practices
  - Python basics, standard library and scientific ecosystem
  - Algorithms and data structures
  - Machine learning
  - Specific tools for high-energy physics or medical physics
- This is not so much about Python or C++—it is about how to write code for effective data analysis
- Will I be a professional data scientist at the end of the semester?
  - No, but hopefully you'll be able to poke around and find the right tool for the job at hand
- Pre-requisites
  - Have a vague idea of how a computer operates
  - If you have ever programmed before that would be great!
  - We shall ask you to fill a form next week to gauge your background and expectations

Basic structure of the course



- Changes with respect to previous years:
  - 5 hours/week instead of 6 (and a much more favorable timetable)
  - Course is *decompressed* in $\sim$ 1.5 semesters (ending in mid April)
  - 9 CFU refactored into three optional modules (ML, HEP, Medical)
  - Note HEP and Medical run in parallel
  - For PhD students we suggest to start from the second part of the Advanced module (more info on e-learning)

- Collaborative tools
  - Version control, development workflow
  - Unit testing, continuous integration, static analysis, documentation
- Python basics
  - Coding conventions, structuring a package
  - Variables, native types, functions
  - The Python standard library
- Algorithms and data structures
  - Complexity and asymptotic running time
  - Python data structures and native algorithms
- Object-Oriented Programming (OOP)
  - Classes, inheritance, composition
  - Operator overload and emulation of Python builtin types
  - Errors, exceptions, iterators and generators, decorators
- The Python computing ecosystem
  - numpy: arrays, functions, broadcasting, vectorization
  - scipy and pandas

- Advanced numpy
- Parallel computing
  - Computer architectures, memory, scaling laws, CPUs and GPUs
  - Parallel programming: concurrency and parallelism, threading in Python
- Introduction to machine learning
  - Classification and regression: boosted decision trees and multilayer perceptrons
  - Deep learning: neural networks, the keras library
  - Supervised and unsupervised training, reinforcement learning
  - Feed Forward and Convolutional architectures

- Modern Deep Networks architectures
- The PyTorch library
- Generative Models
- Graph Networks
- Attention and Transformers

- Introduction to C++
  - Coding style and organization, declaration of interfaces
  - Classes: constructors, virtual functions, private and public, abstract classes, inheritance
  - References, pointers, dynamic memory allocation, memory ownership, smart pointers
  - Templates, standard template library
  - C++11 and C++14: lambda functions, auto variables
- More parallel computing
  - Cuda and OpenCL
  - Examples of algorithms for HEP
  - GPU in HEP Data Analysis
- The ROOT data analysis framework
  - ROOT toolkit
  - PyROOT, root-numpy, RDataFrame

- Medical data processing and feature extraction (python/MATLAB)
  - Tools for handling standard-format medical data (DICOM)
  - Data anonymization and visualization
  - Deriving features form images, image segmentation
  - Data quality control pipelines: outlier removal, dimensionality reduction
- Data analysis and classification (python/MATLAB)
  - Performance evaluations: figures of merit, cross-validation schemes, permutation test
  - Machine-learning and deep-learning tools for segmentation and classification
  - Data augmentation, transfer learning, retrieving localization information.



Logistics

Timetable and final exam

- e-learning: `https://elearning.df.unipi.it/course/view.php?id=344`
- Timetable: 5 hours a week
  - Tuesday, 08:30–11:30 (M-Lab), Friday, 16:30–18:30 (I-Lab)
  - If everybody agrees: start at *:30 sharp(-ish), one 15-minutes break, finish 15 minutes early
- Final exam
  - Development of a specific, reasonable-sized software project
    - We have a list of suggestions on e-learning, but encourage everybody to come up with original projects—if you do so reach out to us well in advance to make sure the project is appropriate
    - Projects can be done individually or in groups of two
  - Two-page description of the project and source code made available $\sim$ 1 week in advance
    - We expect a well-structure repository
  - Oral exam starts with a presentation of the project
    - Aim at 10 slides for 15–20 minutes
  - A few questions on the course material from a pre-compiled list
    - List from last year on e-learning, to be updated
    - We expect the answers to the questions to be thorough and in-depth